

UniTask

UniTask란 ?

- ValueTask를 기반으로 유니티 생명주기에 맞춰 설계된 구조체 비동기 라이브러리

단어 개념 설명

**** 비동기** : 시간이 오래 걸리는 작업(에셋 로드, 네트워크 통신 등)을 백그라운드로 넘겨, 게임 화면이 멈추거나 렉 걸리는 것을 방지하는 것

**** Task** : C# .NET에 내장된 기본적인 비동기 도구(Class = 참조타입), 고로 호출할 때마다 힙 메모리에 객체를 새로 생성해 GC를 유발

**** ValueTask** : Task의 GC문제를 줄이기 위해 구조체(값 타입)기반 비동기 타입. 대기 없이 즉시 완료되는 상황에서는 GC가 없지만, 대기가 발생하면 내부적으로 다시 Task를 생성

사용하는 이유

- 메모리 최적화 : 구조체 기반 설계, 자체적으로 오브젝트 풀링 기능을 사용 => 비동기 호출이 잦아도 GC에 부하를 주지 않아 프레임 드랍 차단
- 유니티 생명주기에 맞춰서 작동 : Unity 환경 **Update**, **FixedUpdate**, **LateUpdate** 등에서 안전하게 제어
- 생산성, 가독성 향상 :
 - 코루틴과 달리 비동기 처리의 결과값(**UniTask<T>**)을 직관적으로 반환받을 수 있음
 - **await** 문법을 사용해 콜백없이 코드 작성
 - 오브젝트 파괴 시 비동기 작업을 자동으로 안전하게 취소 처리(**CancellationToken**)

설치 및 설정

- 프로젝트 파일 내부에 Packages폴더 -> manifest.json파일 열기 -> "dependencies": {} 괄호 안에
"com.cysharp.unitask": "https://github.com/Cysharp/UniTask.git?
path=src/UniTask/Assets/Plugins/UniTask"복붙(붙여넣는 위치에 따라 콤마 위치 주의)

사용 예시

1. 시간 대기, 프레임 제어 (Delay & Yield)

- 코루틴의 `WaitForSeconds`, `yield return null`을 대체하며 가비지가 발생하지 않음

```
using Cysharp.Threading.Tasks;          // 네임스페이스 필요

public class FrameControlExample : MonoBehaviour
{
    public async UniTask ExampleAsync()
    {
        // 1초(1000ms) 대기 (기본값은 게임 시간 기준)
        await UniTask.Delay(1000);

        // 일시정지(Time.timeScale = 0) 상태에서도 현실 시간 기준으로 1초 대기
        await UniTask.Delay(TimeSpan.FromSeconds(1), delayType:
DelayType.Realtime);

        // 다음 Update 프레임까지 대기 (코루틴의 yield return null 대체)
        await UniTask.Yield();

        // 특정 유니티 생명주기 시점까지 대기 (물리 연산 직전까지 대기)
        await UniTask.Yield(PlayerLoopTiming.FixedUpdate);
    }
}
```

2. 결과값 반환 비동기 메서드

- 비동기 작업이 끝난 후 데이터를 안전하게 직접 리턴받을 수 있어서 코루틴의 콜백 방식보다 가독성이 좋음

```
// 호출부 예시
private async UniTaskVoid Start()
{
    // 비동기 함수의 리턴값을 동기 함수처럼 직관적으로 변수에 대입
    int finalScore = await GetScoreFromServerAsync();
    Debug.Log($"서버에서 받아온 유저 점수: {finalScore}");
}

// 생성부 예시: 리턴할 데이터 타입을 <T>에 명시
public async UniTask<int> GetScoreFromServerAsync()
{
    // 가상의 네트워크 딜레이 1.5초 시뮬레이션
    await UniTask.Delay(1500);

    int dummyScore = 100;
    return dummyScore;
}
```

3. 오브젝트 파괴 시 자동 취소

- 비동기 로드 중 오브젝트가 파괴되면(씬 이동, UI닫기 등) 로직을 안전하게 중단시켜 메모리 누수와 널 레퍼런스 에러 방지

```
private async UniTaskVoid Start()
{
    // 이 컴포넌트가 파괴되면 자동으로 취소 신호를 보내는 토큰 준비
    var cancellationToken = this.GetCancellationTokenOnDestroy();

    try
    {
        Debug.Log("5초 대기 시작...");
        // 대기 메서드에 토큰을 매개변수로 반드시 전달
        await UniTask.Delay(5000, cancellation_token: cancellationToken);

        // 만약 5초가 지나기 전에 오브젝트가 파괴되면 아래 코드는 실행되지 않습니다.
        Debug.Log("5초 대기 정상 완료!");
    }
    catch (OperationCanceledException)
    {
        // 오브젝트 파괴로 인해 작업이 안전하게 취소되었을 때 진입 (생략 가능)
        Debug.Log("오브젝트가 파괴되어 비동기 작업이 안전하게 취소되었습니다.");
    }
}
```

4. 복수 작업 동시 처리

- 여러 개의 비동기 작업을 각각 기다리지 않고, 동시에 병렬로 실행시켜 총 소요 시간을 단축

```
public async UniTask LoadAllDataAsync()
{
    // 두 작업을 실행만 시키고 await로 대기하지 않음 (병렬 시작)
    UniTask task1 = LoadPlayerModelAsync();
    UniTask task2 = LoadMapDataAsync();

    // 두 개의 작업이 모두 끝날 때까지 동시 대기
    await UniTask.WhenAll(task1, task2);

    Debug.Log("모든 로딩이 완료되어 게임을 시작합니다.");
}
```

5. Addressables과 연동

- 어드레서블의 비동기 핸들 뒤에 `.ToUniTask()`만 붙이면 연동 됨

```
using UnityEngine;
using Cysharp.Threading.Tasks; // .ToUniTask() 확장 메서드 사용을 위해 필수
using UnityEngine.AddressableAssets; // Addressables 사용을 위해 필수

async UniTaskVoid Start()
{
    // 맵 로딩이 끝날 때까지 코드가 여기서 대기 (그동안 게임 화면은 안 멈춤)
    GameObject mapPrefab = await Addressables.LoadAssetAsync<GameObject>
("Map").ToUniTask();
    Instantiate(mapPrefab);

    // 맵이 생성된 후, 바로 아래 줄에서 플레이어를 부름
    GameObject playerPrefab = await Addressables.LoadAssetAsync<GameObject>
("Player").ToUniTask();
    Instantiate(playerPrefab);

    // 이렇게 어드레서블 문법만 사용하는 것보다 코드 가독성이 좋아짐
}
```

- 하단은 위와 같은 내용을 어드레서블 자체 비동기 방식으로 코드를 작성한 비교 예시

```
void Start()
{
    // 맵 좀 비동기 로드
    Addressables.LoadAssetAsync<GameObject>("Map").Completed += handle =>
    {
        // 다 가져오면 이 중괄호 안쪽이 실행됨
        GameObject mapPrefab = handle.Result;
        Instantiate(mapPrefab);

        // 만약 맵을 만들고 나서, '플레이어 캐릭터'도 이어서 로드해야 한다면?
        Addressables.LoadAssetAsync<GameObject>("Player").Completed +=
playerHandle =>
        {
            // 중괄호 안에 중괄호가 또 들어감
            GameObject playerPrefab = playerHandle.Result;
            Instantiate(playerPrefab);
        };
    };
}
```